

Reviewer Pack — Cubical First-Betti Linearly Lower-Bounds DNF-MCSP

Entry 9020e29f840c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 01:33:54 UTC

Cubical First-Betti Linearly Lower-Bounds DNF-MCSP

Entry ID: 9020e29f840c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 01:33:54 UTC

1. Conjecture

Field A (mathematical branch): Cubical combinatorial topology — the cubical subcube complex $K_f := \{\sigma \subseteq \{0,1\}^n : \sigma \text{ is a subcube and } \sigma \subseteq f^{-1}(1)\}$ with its boundary maps over F_2 , $\beta_1(K_f)$ computed as $\text{nullity}(\partial_1) - \text{rank}(\partial_2)$ (Kaczynski-Mischaikow-Mrozek 2004 ‘Computational Homology’; Edelsbrunner-Letscher-Zomorodian 2002; Carlsson 2009 ‘Topology and data’). A non-ring, F_2 -rank-of-cubical-boundary functional: ranks of integer cellular boundaries do NOT extend to a polynomial F_q lift of the Boolean ring (the cubical boundary involves signed face inclusion not preserved by polynomial ring lifts), so the bridge is Aaronson-Wigderson algebrization-safe. The invariant is RELATIONAL (a categorical functor $(\text{Bool}^n \rightarrow \text{Top} \rightarrow \text{Vect}\{F_2\})$ applied to the subcube cover poset), not a single truth-table number — sidestepping trap (j). An arXiv search ‘cubical homology’ AND (‘MCSP’ OR ‘minimum DNF’ OR ‘meta-complexity’) returns 0 direct hits with <5 adjacent papers (all on TDA of biological data or persistent-homology classifiers, never on DNF-MCSP). Distinct from blacklisted Mahler-measure (single complex-analytic truth-table number), FCA antichain width (Galois-lattice width on assignments $\times$ literals), treewidth of PI compatibility graph (graph-minor invariant), and independence-complex Euler $\tilde{\chi}$ (Stanley-Reisner alternating sum). **Field B** (complexity object): DNF-MCSP / minimum DNF size $\sigma_{\text{DNF}}(f)$ for $f: \{0,1\}^n \rightarrow \{0,1\}$ in the Quine-McCluskey / Masek 1979 / Umans 1999 / Allender-Hellerstein-McCabe-Pitassi-Saks / Hirahara-Oliveira meta-complexity regime, where σ_{DNF} is the optimum of a prime-implicant set-cover and the canonical NP-hard meta-complexity object whose hardness fuels Hirahara’s worst-case-to-average-case program. Sparsity escape from Razborov-Rudich: ‘ $\sigma_{\text{DNF}}(f) \leq s$ ’ is rare among random f (only $\sim 2^{O(s \log n)}$ functions admit it, of $2^{\Omega(n)}$ total), so any lower bound $\sigma_{\text{DNF}} \geq \varphi(f)$ inherits sparsity from the cap side and does not violate Razborov-Rudich largeness.

Statement:

Let $K_f = \{\sigma \subseteq \{0,1\}^n : \sigma \text{ is a subcube and } \sigma \subseteq f^{-1}(1)\}$ be the cubical subcube complex of $f^{-1}(1)$, and $\beta_1 := \beta_1(K_f, F_2)$ its first F_2 -Betti number. Then for every Boolean $f: \{0,1\}^n \rightarrow \{0,1\}$ satisfying $\beta_1(K_f) \geq 1$, the minimum DNF size obeys $\sigma_{\text{DNF}}(f) \geq 3 \cdot \beta_1(K_f)$. One counterexample (a single f with $\sigma_{\text{DNF}}(f) < 3 \cdot \beta_1(K_f)$ and $\beta_1(K_f) \geq 1$) refutes the conjecture.

Rationale (proposer’s reasoning):

By the nerve theorem applied to any DNF cover $F = T_1 \vee \dots \vee T_s$ of f (each T_i a subcube $\subseteq f^{-1}(1)$), the nerve N_F is homotopy equivalent to K_f , giving $\beta_1(N_F) = \beta_1$. Trivially N_F has s vertices and any 1-cycle uses ≥ 3 of them, yielding only $\sigma_{\text{DNF}} \geq \sqrt{2\beta_1}$; the conjecture asserts the much stronger LINEAR bound — each independent topological

hole costs 3 fresh terms in any optimal cover because subcubes encircling a hole are ‘rigid’ (cannot be shared across distinct holes without collapsing the cycle). This is a clean nerve-rigidity strengthening pioneered by no prior MCSP work.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6692915bd1cd5a6b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all instances with $\beta_1(K_f) \geq 1$ from 30 seeds $\times n \in \{4,5,6\} \times p \in \{0.30,0.50,0.70\}$ (≤ 270 total), the conjecture is SUPPORTED iff every filtered instance satisfies $\sigma_{DNF}(f) \geq 3 \cdot \beta_1(K_f)$; a single instance with $\sigma_{DNF}(f) < 3 \cdot \beta_1(K_f)$ FALSIFIES it.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - cubical homology Boolean function DNF minimum implicant - first Betti number subcube complex meta-complexity MCSP - F₂ Betti number prime implicant cover lower bound DNF

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from itertools import product, combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_subcube(f, sub):
        return all(f(tuple(sub)) == 1 for sub in product(*[range(bit + 1) for bit in sub]))

    def dnf_size(f, n):
        K_f = [tuple(sub) for sub in product(range(2), repeat=n) if is_subcube(f, sub)]
        beta_1 = compute_beta_1(K_f)
```

```

sigma_DNF = compute_sigma_DNF(f, n)
return beta_1, sigma_DNF

def compute_beta_1(K_f):
    n = len(K_f[0])
    F2_matrix = [[0] * (2 ** n) for _ in range(2 ** n)]
    for i, subcube in enumerate(K_f):
        for j in range(n):
            new_subcube = list(subcube)
            new_subcube[j] = 1 - new_subcube[j]
            F2_matrix[i][K_f.index(tuple(new_subcube))] = 1
    return gaussian_elimination(F2_matrix, n)

def gaussian_elimination(matrix, n):
    rows, cols = len(matrix), len(matrix[0])
    rank = 0
    for col in range(cols):
        pivot_row = -1
        for row in range(rank, rows):
            if matrix[row][col] == 1:
                pivot_row = row
                break
        if pivot_row != -1:
            matrix[pivot_row], matrix[rank] = matrix[rank], matrix[pivot_row]
            rank += 1
            for row in range(rows):
                if row != rank - 1 and matrix[row][col] == 1:
                    matrix[row] = [matrix[row][i] ^ matrix[rank - 1][i] for i in range(cols)]
    return cols - rank

def compute_sigma_DNF(f, n):
    prime_implicants = []
    for subcube in product(range(2), repeat=n):
        if f(subcube) == 1:
            prime_implicants.append(subcube)
    cover = set()
    for implicant in prime_implicants:
        uncovered = [i for i in range(n) if any(implicant[i] != cube[i] for cube in cover)]
        if not uncovered:
            continue
        min_cover = []
        for bit in uncovered:
            new_cover = cover.copy()
            new_cover.add(bit)
            min_cover.append(new_cover)
        cover = min(min_cover, key=len)
    return len(cover)

n_values = [4, 5, 6]
p_values = [0.30, 0.50, 0.70]
instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for p in p_values:

```

```

    for _ in range(30):
        f = lambda x: random.choices([0, 1], [1 - p, p])[0]
        beta_1, sigma_DNF = dnf_size(f, n)
        if beta_1 >= 1 and sigma_DNF < 3 * beta_1:
            conjecture_holds = False
            counterexample = f"f with n={n}, p={p}"
            break
    instances_tested += 30

    return {
        "metric_name": "sigma_DNF",
        "metric_value": 3 * compute_beta_1(K_f),
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []

    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        results.append(trial)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = (sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results)) ** 0.5
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\", first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_80da2c5c.py", line 105, in <module>
    trial = run_trial(seed)
            ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_80da2c5c.py", line 84, in run_trial
    beta_1, sigma_DNF = dnf_size(f, n)
                        ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_80da2c5c.py", line 25, in dnf_size
    beta_1 = compute_beta_1(K_f)

```

```

^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_80da2c5c.py", line 30, in compute_beta_
    n = len(K_f[0])
    ~~~^^^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in compute_beta_1 (empty K_f list) before any instances were evaluated, so no data was produced to assess the pre-registered support or falsification condition. | next: Fix compute_beta_1 to handle the empty subcube complex case ($\beta_1 = 0$ when K_f is empty) and rerun the 270-triple sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	425304
2	preregistration	claude_max	opus	0	0	5464
3	novelty	claude_max	opus	0	0	2809
4	novelty	claude_max	opus	0	0	6634
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14087
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14182
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13777
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13369
9	judge	claude_max	opus	0	0	4697

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 500323 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in research/audit/9020e29f840c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9020e29f840c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9020e29f840c.tar.gz (if generated)